

PROJECT AND TEAM INFORMATION

Project Title

AdaptiveOS: A Database-Integrated Historical Workload Intelligence Framework for Adaptive OS Resource Management

Student / Team Information

Team Name:	#T001 - Envision
Team member 1 (Team Lead)	 Rawat, Priyanshi-24012530 priyanshirawat316@gmail.com
Team member 2	 Singh Kushwah, Shashwat – 230211069 shashwatk@gmail.com
Team member 3	 Yadav , Abhishek-23021986 a38376718@gmail.com

PROPOSAL DESCRIPTION

Motivation (1 pt)

Operating systems must continuously decide how to allocate CPU time, memory, and other resources among competing processes. Classical scheduling algorithms such as FCFS, SJF, Priority Scheduling, and Round Robin are typically applied as fixed, general-purpose strategies, yet workloads differ substantially in their CPU, memory, and I/O demands, and a single policy that performs well for one workload type frequently performs poorly for another. Most introductory OS implementations, and even many production systems, apply a fixed policy regardless of the nature of the incoming workload, discarding the fact that similar workloads tend to recur over time. The motivation for AdaptiveOS is to close this gap by giving the OS a persistent memory of how previous workloads behaved under different resource-management policies, so that future decisions can be grounded in historical evidence rather than a single static rule. This bridges OS resource management and DBMS-backed historical intelligence into one explainable feedback system.

State of the Art / Current solution

Existing educational OS projects typically implement scheduling algorithms in isolation and compute metrics such as average waiting time for a single run, without retaining any memory across executions. Existing DBMS projects typically demonstrate CRUD operations on unrelated tables (students, employees, processes) without those records ever influencing a live decision process. Workload-aware resource management itself is an established research direction: OS scheduling research emphasizes that workload characteristics and performance goals must be considered jointly rather than assuming one policy is universally optimal, and in cloud environments, projects such as Microsoft's Resource Central have shown that historical VM behavior is often consistent across a workload's lifetime and can be used to inform future resource-management decisions at scale. There is also research precedent for favoring lightweight, transparent, historical-data-driven approaches over heavyweight machine-learning models where explainability matters. Our current solution builds on this precedent: rather than inventing workload-aware scheduling as a concept, or reaching for opaque ML models, AdaptiveOS integrates an OS simulator with a relational database that persistently stores workload profiles and outcomes, and uses historical similarity matching a transparent, explainable technique to select policies, falling back to direct evaluation only when no comparable history exists.

Project Goals and Milestones

General Goals:

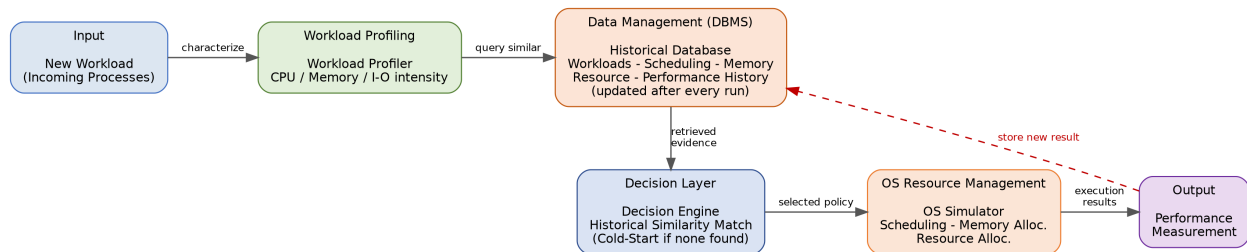
- Develop an OS simulator covering process management, CPU scheduling (FCFS, SJF, Priority, Round Robin), memory allocation (First Fit, Best Fit, Worst Fit), and resource request/allocation/release.
- Design and implement a relational DBMS schema that persists workload characteristics, execution histories, and performance outcomes (WORKLOAD, PROCESS, CPU/MEMORY/IO_USAGE, SCHEDULING/MEMORY/RESOURCE_HISTORY, PERFORMANCE_RESULT).
- Implement a Workload Profiler that characterizes each new workload by CPU/memory/I-O intensity, process count, and average burst time.
- Implement a Decision Engine that queries historical data for similar workloads and selects a resource-management policy based on documented past performance.
- Implement a cold-start mode that evaluates candidate policies directly when no similar historical workload exists, and stores the new result for future reuse.
- Conduct comparative experiments (Fixed Round Robin vs. Fixed SJF vs. AdaptiveOS) measuring waiting time, turnaround time, response time, and utilization.

Project Approach

Steps:

1. **Initialization:** Set up the OS simulator (process manager, CPU scheduler, memory allocator, resource manager) and the DBMS schema with the required historical tables.
2. **Workload Generation & Profiling:** Generate simulated workloads with varying process counts, burst times, and CPU/memory/I-O demands, and compute a workload profile for each.
3. **Historical Query & Decision Routing:** Query the DBMS for historically similar workloads; if sufficient evidence exists, select the policy that previously performed best. If not, enter cold-start mode and evaluate candidate policies directly.
4. **Execution & Feedback Storage:** Execute the workload under the selected policy in the OS simulator, measure performance outcomes, and store the new result back into the DBMS closing the feedback loop so future decisions benefit from this experiment.

System Architecture



A new workload is characterized by the Workload Profiler and compared against historical data stored in the DBMS. The Decision Engine retrieves evidence from similar past workloads (or triggers a cold-start evaluation when none exists) and selects a resource-management policy for the OS Simulator to execute. Performance Measurement records the outcome, which is written back into the DBMS shown as the dashed feedback edge so the historical database becomes more informative with every workload processed.

Project Outcome / Deliverables (1 pts)

The final deliverables will include:

- A functional OS simulator demonstrating process management, CPU scheduling, memory allocation, and resource management.
- A relational DBMS implementation with the full historical schema, including ER model, normalization, and the SQL queries used for historical similarity retrieval.
- A working Workload Profiler and Decision Engine, including the cold-start evaluation logic.
- Source code with inline documentation covering the scheduling, allocation, and historical-matching logic.
- An experimental comparison report (Fixed Round Robin vs. Fixed SJF vs. AdaptiveOS) with measured performance metrics.

Assumptions

- The system is a research prototype / OS simulator intended for educational and experimental purposes, and does not modify or replace a real operating system's scheduler.
- Workloads and processes are simulated programmatically rather than executed as real OS-level threads.
- The historical database is populated through the project's own simulated experiments rather than imported from an external production system.
- Readers/evaluators have a foundational understanding of OS scheduling and relational database concepts sufficient to interpret the historical schema and decision logic.

References

- **Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts*.**
- **Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems*.**
- **Cortez, E., Bonde, A., Muzio, A., et al. (2017). *Resource Central: Understanding and Predicting Workloads for Improved Resource Management in Large Cloud Platforms. SOSP*.**
- **TU Delft OpenCourseWare — *Operating Systems Scheduling: Workload and Performance-Goal Considerations*.**